

Operating System Review 2

Chapter 6 to 9

Chapter 6: Process Synchronization

Producer/Consumer Problem

• Producer

```
item nextProduced;
while( true ) {
    /* Produce an item and store it in nextProduced */
    nextProduced = makeNewItem( . . . );

    /* Wait for space to become available */
    while( ( ( in + 1 ) % BUFFER_SIZE ) == out )
        ; /* Do nothing */

    /* And then store the item and repeat the loop. */
    buffer[ in ] = nextProduced;
    in = ( in + 1 ) % BUFFER_SIZE;
}
```

Producer/Consumer Problem

• Consumer

```
item nextConsumed;
while( true ) {
    /* Wait for an item to become available */
    while( in == out )
        ; /* Do nothing */
    /* Get the next available item */
    nextConsumed = buffer[ out ];
    out = ( out + 1 ) % BUFFER_SIZE;

    /* Consume the item in nextConsumed
       ( Do something with it ) */

}
```

Producer/Consumer Problem (why not use counter?)

Producer Process:

```
while (true)
{
    /* produce an item in nextProduced */
    while (counter == BUFFER_SIZE)
        ; /* do nothing */
    buffer[in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

Race Starts!

Consumer Process:

```
while (true)
{
    while (counter == 0)
        ; /* do nothing */
    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
    /* consume the item in nextConsumed */
}
```

Arbitrary Result

Producer:

```
register1 = counter  
register1 = register1 + 1  
counter = register1
```

Consumer:

```
register2 = counter  
register2 = register2 - 1  
counter = register2
```

Interleaving:

T_0 :	producer	execute	$register_1 = counter$	{ $register_1 = 5$ }
T_1 :	producer	execute	$register_1 = register_1 + 1$	{ $register_1 = 6$ }
T_2 :	consumer	execute	$register_2 = counter$	{ $register_2 = 5$ }
T_3 :	consumer	execute	$register_2 = register_2 - 1$	{ $register_2 = 4$ }
T_4 :	producer	execute	$counter = register_1$	{ $counter = 6$ }
T_5 :	consumer	execute	$counter = register_2$	{ $counter = 4$ }

Critical Section

1 **Mutual Exclusion**

2 **Progress** - If a process wants to enter its critical section, it must be able to do so without being delayed indefinitely. (No process can be in its remainder section indefinitely.)

3 **Bounded Waiting** - A process requesting to enter its critical section must wait for a bounded number of requests before it can enter. (No process can request to enter its critical section and wait indefinitely.)

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (TRUE);
```

their critical section.

and one or more

is not in their

cannot be postponed

into their critical

es can get into their

and before that

ction will get a turn

go first.)

Synchronization Hardware

```
boolean TestAndSet(boolean *target) {  
    boolean rv = *target;  
    *target = TRUE;  
    return rv;  
}
```

Figure 5.3 The definition of the TestAndSet() instruction.

```
do {  
    while (TestAndSetLock(&lock))  
        ; // do nothing  
  
    // critical section  
  
    lock = FALSE;  
  
    // remainder section  
}while (TRUE);
```

Figure 5.4 Mutual-exclusion implementation with TestAndSet().

Synchronization Hardware

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
  
    return temp;  
}
```

Figure 5.5 The definition of the `compare_and_swap()` instruction.

```
do {  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = 0;  
  
    /* remainder section */  
} while (true);
```

Figure 5.6 Mutual-exclusion implementation with the `compare_and_swap()` instruction.

Semaphores

Wait:

```
wait(S) {  
    while S <= 0  
        ; // no-op  
    S--;  
}
```

Signal:

```
signal(S) {  
    S++;  
}
```

```
do {  
    waiting(mutex);  
  
    // critical section  
  
    signal(mutex);  
  
    // remainder section  
}while (TRUE);
```

Semaphores Implementation

Semaphore Structure:

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

Wait Operation:

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

Signal Operation:

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

Bounded Buffer Problem

```
do {  
    . . .  
    // produce an item in nextp  
    . . .  
    wait(empty);  
    wait(mutex);  
    . . .  
    // add nextp to buffer  
    . . .  
    signal(mutex);  
    signal(full);  
}while (TRUE);
```

Figure 5.9 The structure of the producer process.

```
do {  
    wait(full);  
    wait(mutex);  
    . . .  
    // remove an item from buffer to nextc  
    . . .  
    signal(mutex);  
    signal(empty);  
    . . .  
    // consume the item in nextc  
    . . .  
}while (TRUE);
```

Readers-Writer Problem

```
do {  
    wait(rw_mutex);  
    . . .  
    /* writing is performed */  
    . . .  
    signal(rw_mutex);  
} while (true);
```

```
do {  
    wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        wait(rw_mutex);  
    signal(mutex);  
    . . .  
    /* reading is performed */  
    . . .  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(rw_mutex);  
    signal(mutex);  
} while (true);
```

The Dining-Philosophers Problem

不能只用信号量解决, 需要monitor来解决, monitor不作为考试内容



```
do {  
    wait(chopstick[i]);  
    wait(chopstick[(i+1) % 5]);  
    . . .  
    // eat  
    . . .  
    signal(chopstick[i]);  
    signal(chopstick[(i+1) % 5]);  
    . . .  
    // think  
    . . .  
}while (TRUE);
```

Monitor

```
monitor monitor name
{
    // shared variable declarations

    procedure P1 ( . . . ) {
        . . .
    }

    procedure P2 ( . . . ) {
        . . .
    }

    .
    .
    .
    procedure Pn ( . . . ) {
        . . .
    }

    initialization code ( . . . ) {
        . . .
    }
}
```

The Dining-Philosophers Problem (using Monitor)

```
monitor DiningPhilosophers
{
    enum {THINKING, HUNGRY, EATING} state[5];
    condition self[5];

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait();
    }

    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}
```

```
void test(int i) {
    if ((state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING)) {
        state[i] = EATING;
        self[i].signal();
    }
}

initialization_code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
```

Transactions

T_0	T_1
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)

T_0	T_1
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

Timestamp Scheduling

- Suppose transaction T_i issues a read on data item Q :
 - If $TS(T_i) < \text{the W-timestamp for } Q$, then it is attempting to read data that has been changed. Transaction T_i is rolled back, and restarted with a new timestamp.
 - If $TS(T_i) > \text{the W-timestamp for } Q$, then the R-timestamp for Q is updated to the later of its current value and $TS(T_i)$.
- Suppose T_i issues a write on Q :
 - If $TS(T_i) < \text{the R-timestamp for } Q$, then it is attempting to change data that has already been read in its unaltered state. T_i is rolled back and restarted with a new timestamp.
 - If $TS(T_i) < \text{the W-timestamp for } Q$ it is also rolled back and restarted, for similar reasons.
 - Otherwise, the operation proceeds, and the W-timestamp for Q is updated to $TS(T_i)$.

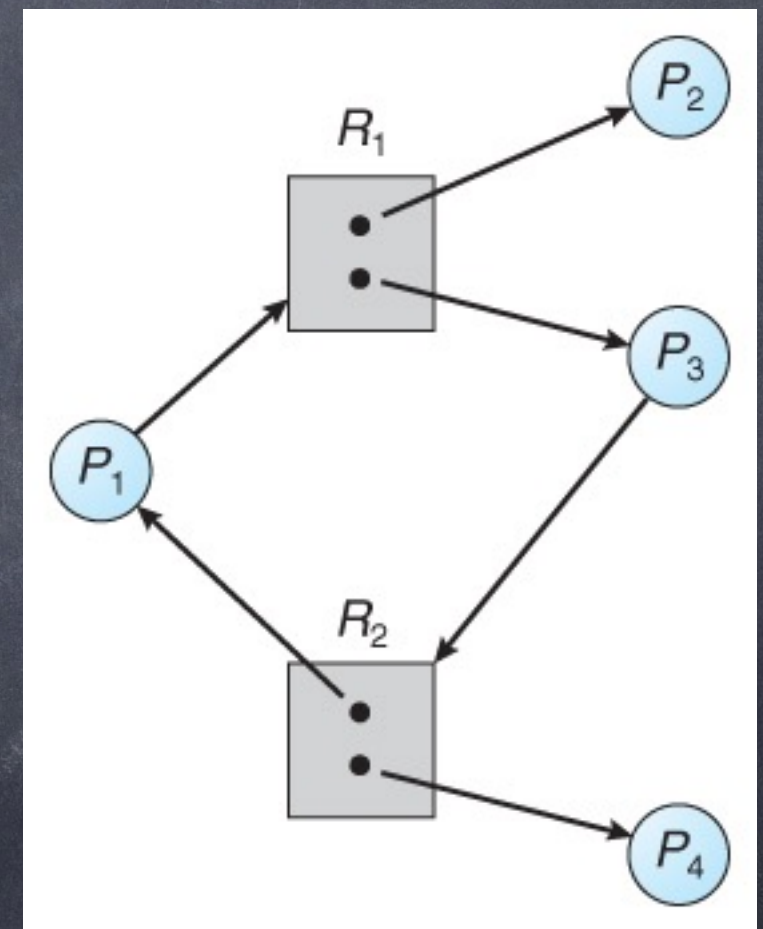
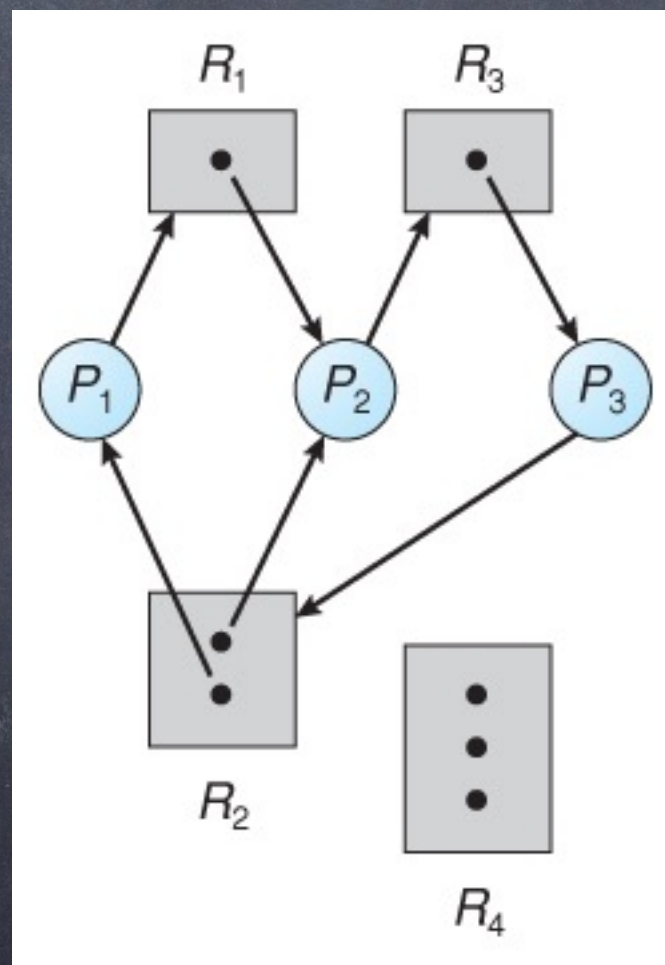
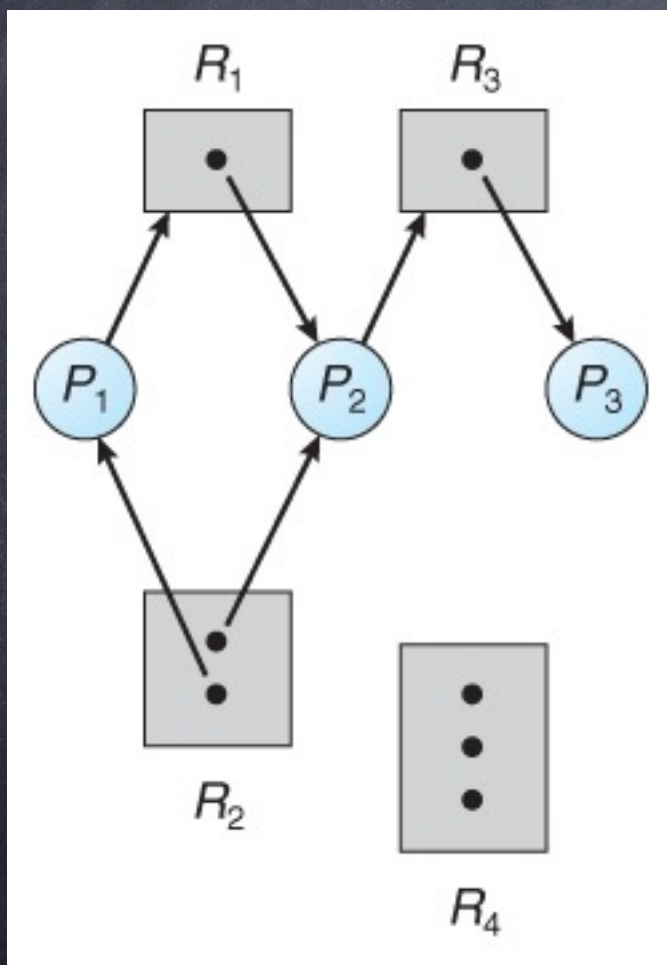
Chapter 7 Deadlock

Four Conditions

- 1 Mutual Exclusion** - At least one resource must be held in a non-sharable mode; If any other process requests this resource, then that process must wait for the resource to be released.
- 2 Hold and Wait** - A process must be simultaneously holding at least one resource and waiting for at least one resource that is currently being held by some other process.
- 3 No preemption** - Once a process is holding a resource (i.e. once its request has been granted), then that resource cannot be taken away from that process until the process voluntarily releases it.
- 4 Circular Wait** - A set of processes $\{ P_0, P_1, P_2, \dots, P_N \}$ must exist such that every $P[i]$ is waiting for $P[(i + 1) \% (N + 1)]$. (Note that this condition implies the hold-and-wait condition, but it is easier to deal with the conditions if the four are considered separately.)

Resource-Allocation Graph

有环不一定死锁



Deadlock Prevention

Mutual Exclusion

- Shared resources such as read-only files do not lead to deadlocks.
- Unfortunately some resources, such as printers and tape drives, require exclusive access by a single process.

Hold and Wait

- To prevent this condition processes must be prevented from holding one or more resources while simultaneously waiting for one or more others. There are several possibilities for this:
 - Require that all processes request all resources at one time. This can be wasteful of system resources if a process needs one resource early in its execution and doesn't need some other resource until much later.
 - Require that processes holding resources must release them before requesting new resources, and then re-acquire the released resources along with the new ones in a single new request. This can be a problem if a process has partially completed an operation using a resource and then fails to get it re-allocated after releasing it.

No Preemption

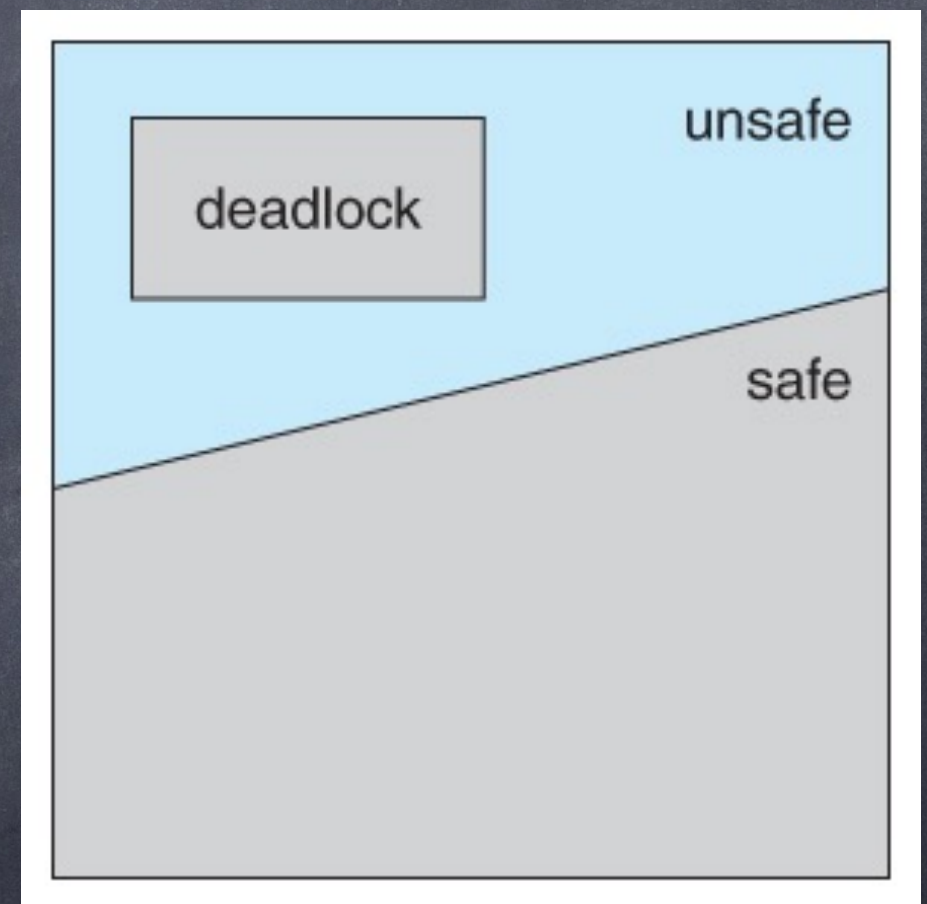
- Preemption of process resource allocations can prevent this condition of deadlocks, when it is possible.

Circular Wait

- One way to avoid circular wait is to number all resources, and to require that processes request resources only in strict order

Deadlock Avoidance

- A state is *safe* if the system can allocate all resources requested by all processes (up to their stated maximums) without entering a deadlock state.
- More formally, a state is safe if there exists a *safe sequence* of processes $\{ P_0, P_1, P_2, \dots, P_N \}$ such that all of the resource requests for P_i can be granted using the resources currently allocated to P_i and all processes P_j where $j < i$. (I.e. if all the processes prior to P_i finish and free up their resources, then P_i will be able to finish also, using the resources that they have freed up.)



Is it safe?

Total = 12

	Maximum Needs	Current Allocation
P0	10	5
P1	4	2
P2	9	2

Safety Algorithm

1. Let **Work** and **Finish** be vectors of length m and n , respectively.
Initialize:

$Work = Available$

$Finish[i] = false$ for $i = 0, 1, \dots, n-1$.

2. Find an i such that both:

(a) $Finish[i] = false$

(b) $Need_i \leq Work$

If no such i exists, go to step 4.

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2.

4. If $Finish[i] == true$ for all i , then the system is in a safe state.

Resource-request Algorithm (bankers algorithm)

反复调用safe 算法

Request = request vector for process P_i . If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j .

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim.

2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available.

3. Pretend to allocate requested resources to P_i by modifying the state as follows:

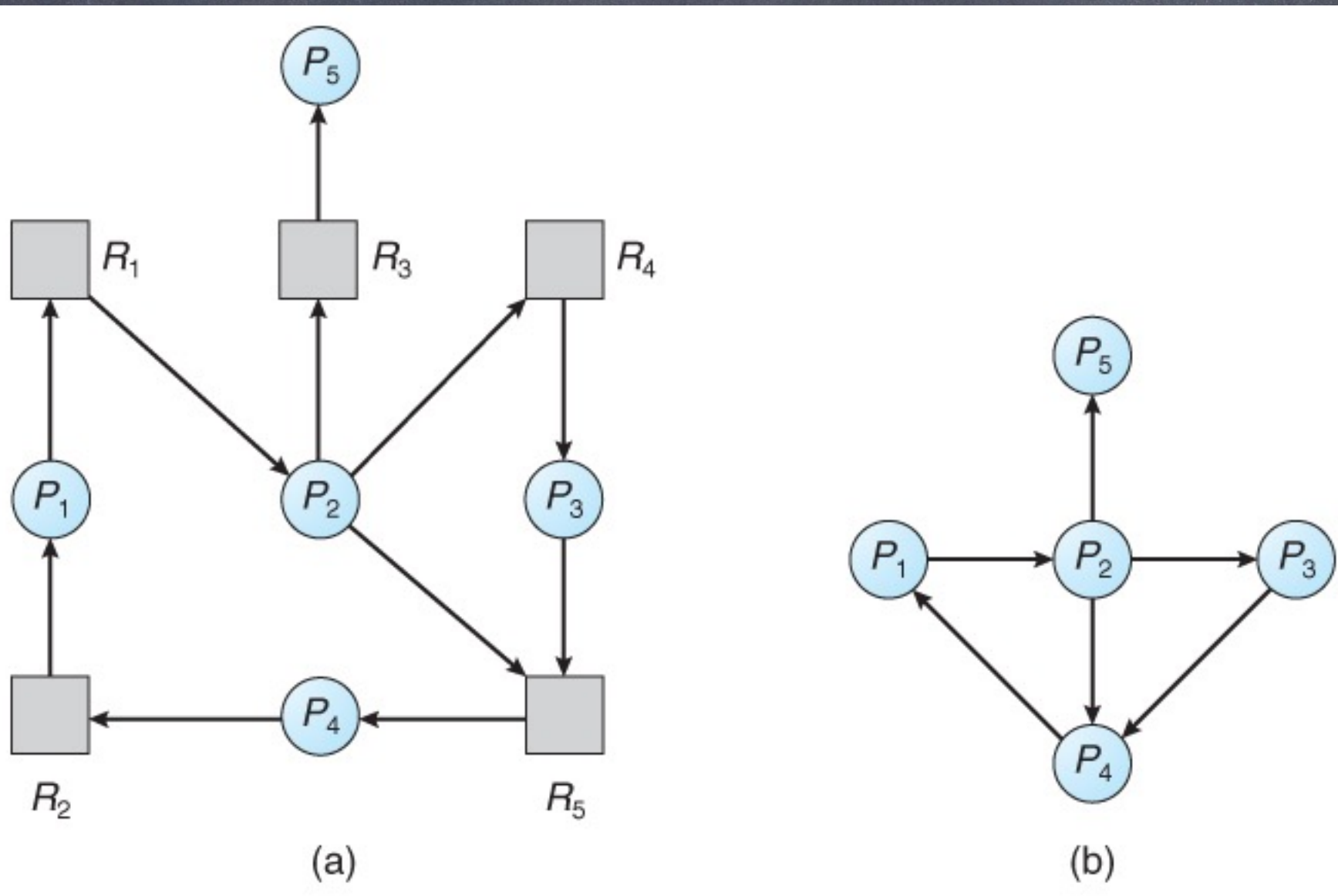
$Available = Available - Request_i$

$Allocation_i = Allocation_i + Request_i$

$Need_i = Need_i - Request_i$

- If safe $\Rightarrow$ the resources are allocated to P_i .
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

Deadlock Detection



Recovery from Deadlock

考试没法考, 没法打分

Process Termination

- Two basic approaches, both of which recover resources allocated to terminated processes:
 - Terminate all processes involved in the deadlock. This definitely solves the deadlock, but at the expense of terminating more processes than would be absolutely necessary.
 - Terminate processes one by one until the deadlock is broken. This is more conservative, but requires doing deadlock detection after each step.

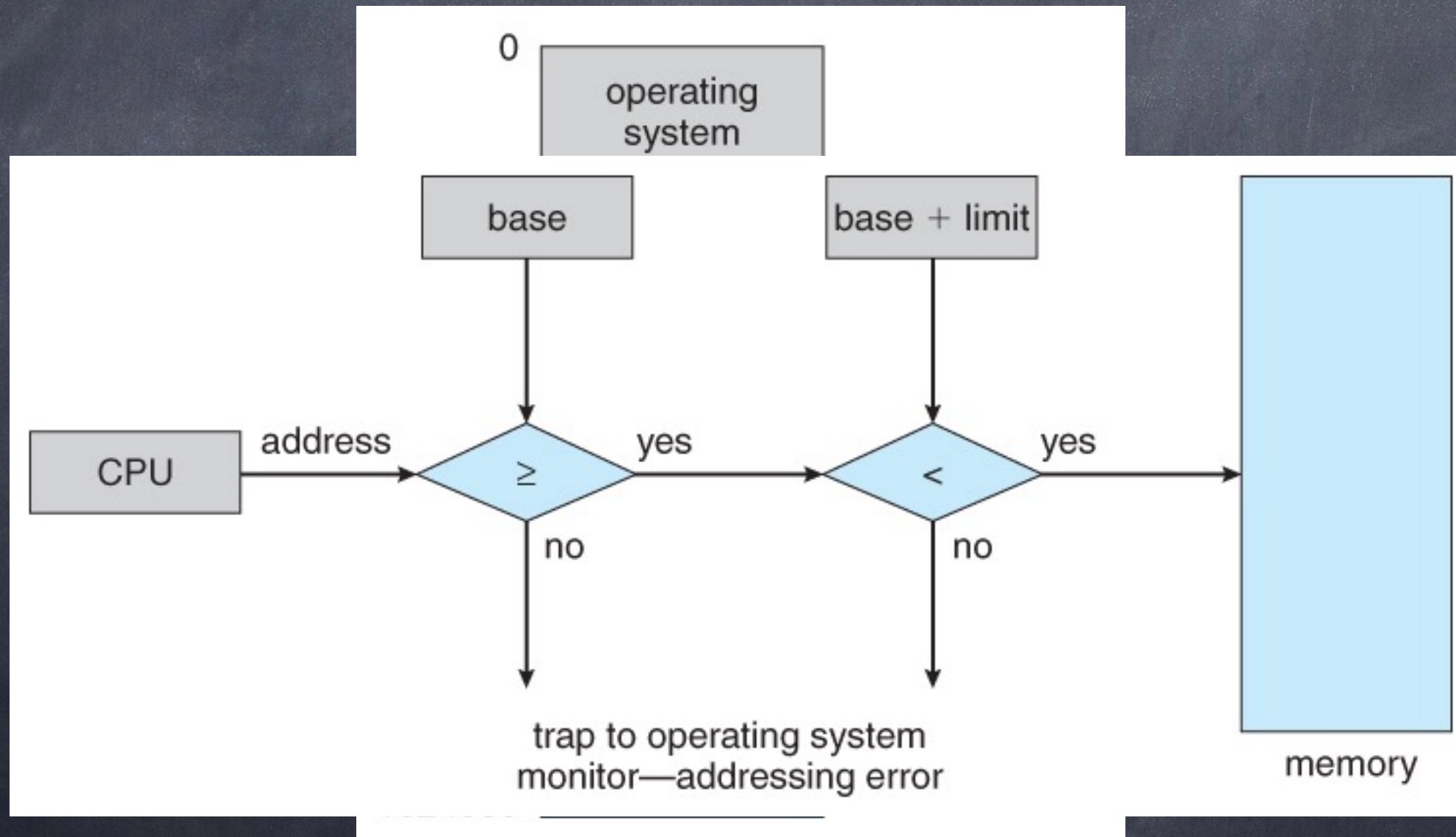
Resource Preemption

- When preempting resources to relieve deadlock, there are three important issues to be addressed:
 - 1 Selecting a victim** - Deciding which resources to preempt from which processes involves many of the same decision criteria outlined above.
 - 2 Rollback** - Ideally one would like to roll back a preempted process to a safe state prior to the point at which that resource was originally allocated to the process. Unfortunately it can be difficult or impossible to determine what such a safe state is, and so the only safe rollback is to roll back all the way back to the beginning. (I.e. abort the process and make it start over.)

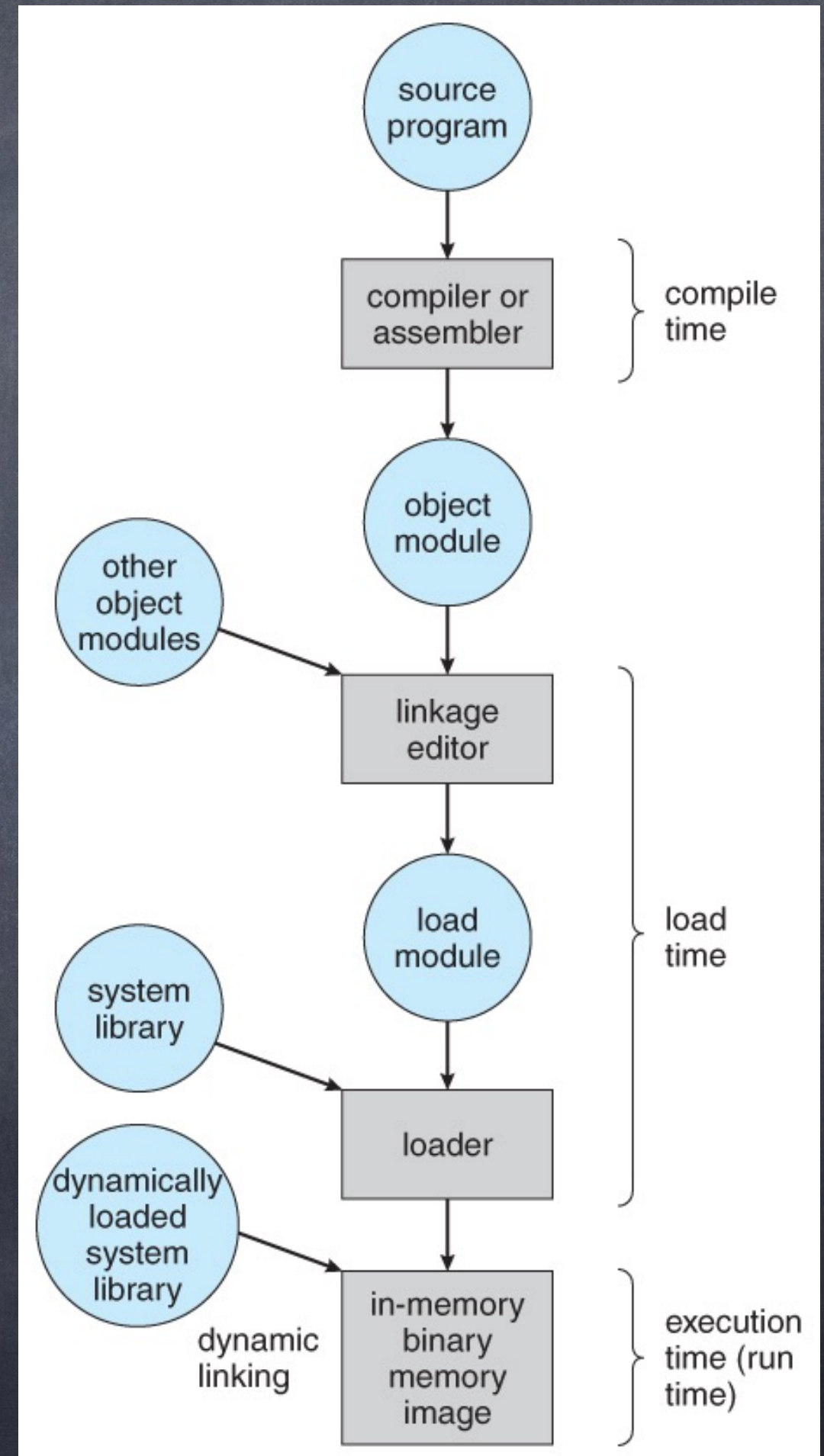
Chapter 8-9

Memory

Basic Idea



Address Binding



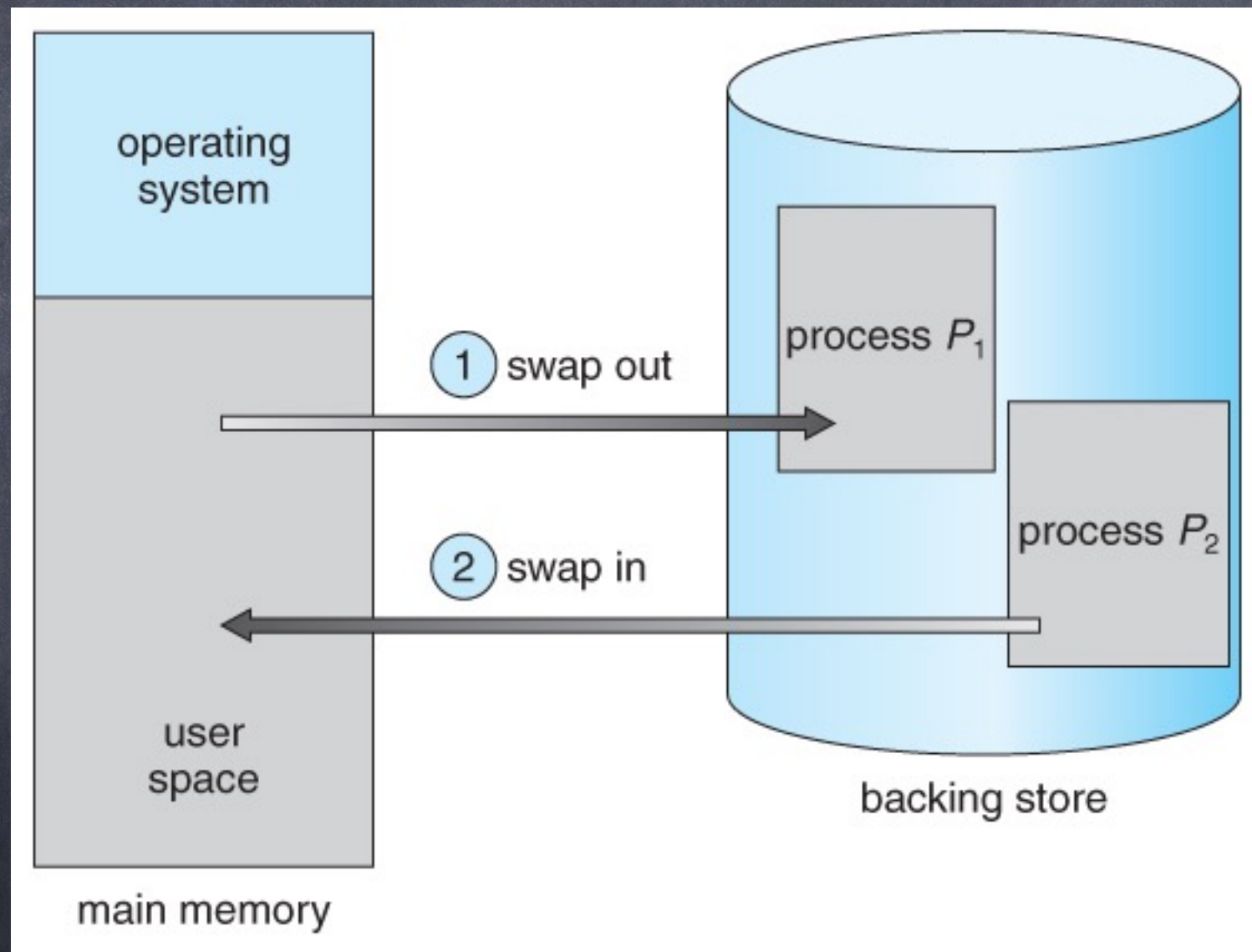
Dynamic Loading and Linking

Rather than loading an entire program into memory at once, dynamic loading loads up each routine as it is called. The advantage is that unused routines need never be loaded, reducing total memory usage and generating faster program startup times. The downside is the added complexity and overhead of checking to see if a routine is loaded every time it is called and then then loading it up if it is not already loaded.

dynamic linking, however, only a stub is linked into the executable module, containing references to the actual library module linked in at run time.

- This method saves disk space, because the library routines do not need to be fully included in the executable modules, only the stubs.

Swapping

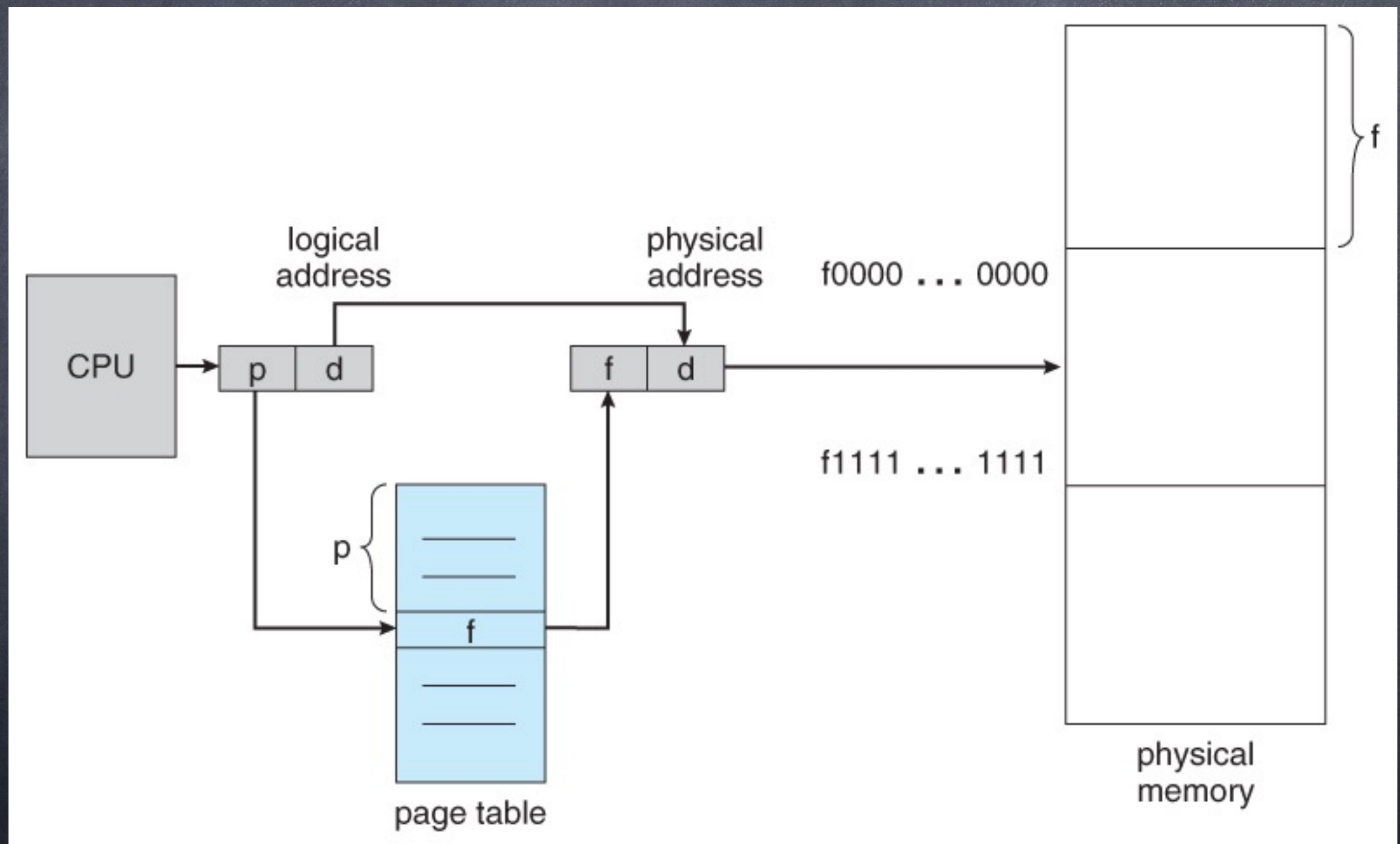


Memory Allocation

An alternate approach is to keep a list of unused (free) memory blocks (holes), and to find a hole of a suitable size whenever a process needs to be loaded into memory. There are many different strategies for finding the "best" allocation of memory to processes, including the three most commonly discussed:

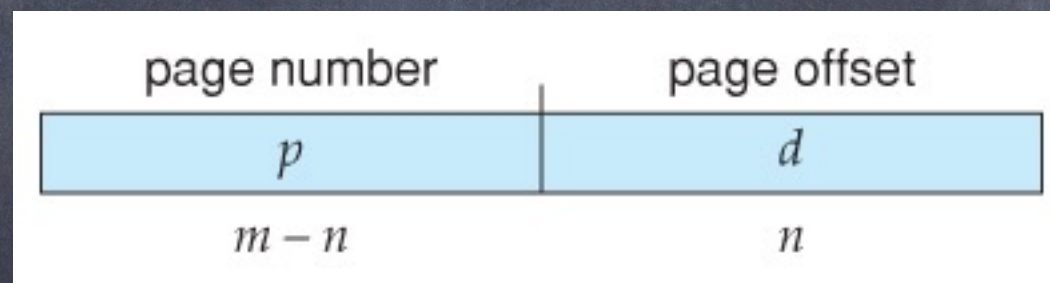
- 1 **First fit** - Search the list of holes until one is found that is big enough to satisfy the request, and assign a portion of that hole to that process. Whatever fraction of the hole not needed by the request is left on the free list as a smaller hole. Subsequent requests may start looking either from the beginning of the list or from the point at which this search ended.
- 2 **Best fit** - Allocate the *smallest* hole that is big enough to satisfy the request. This saves large holes for other process requests that may need them later, but the resulting unused portions of holes may be too small to be of any use, and will therefore be wasted. Keeping the free list sorted can speed up the process of finding the right hole.
- 3 **Worst fit** - Allocate the largest hole available, thereby increasing the likelihood that the remaining portion will be usable for satisfying future requests.

Paging



Page Size

Page numbers, frame numbers, and frame sizes are determined by the architecture, but are typically powers of two, allowing addresses to be split at a certain number of bits. For example, if the logical address size is 2^m and the page size is 2^n , then the high-order $m-n$ bits of a logical address designate the page number and the remaining n bits represent the offset.

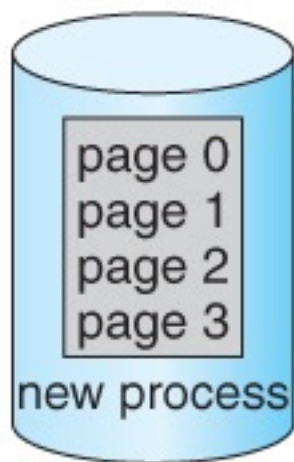


Page table entries (frame numbers) are typically 32 bit numbers, allowing access to 2^{32} physical page frames. If those frames are 4 KB in size each, that translates to 16 TB of addressable physical memory. ($32 + 12 = 44$ bits of physical address space.)

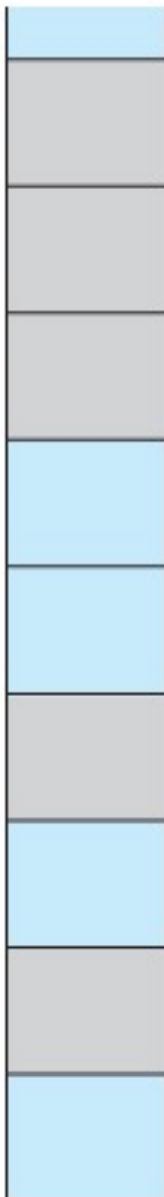
Page Allocation

free-frame list

14
13
18
20
15



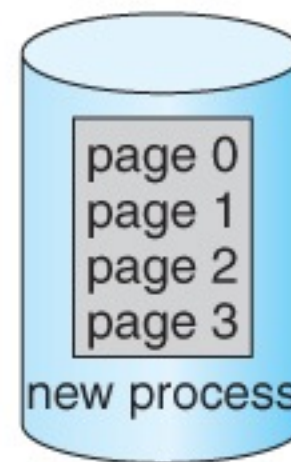
13
14
15
16
17
18
19
20
21



(a)

free-frame list

15



0	14
1	13
2	18
3	20

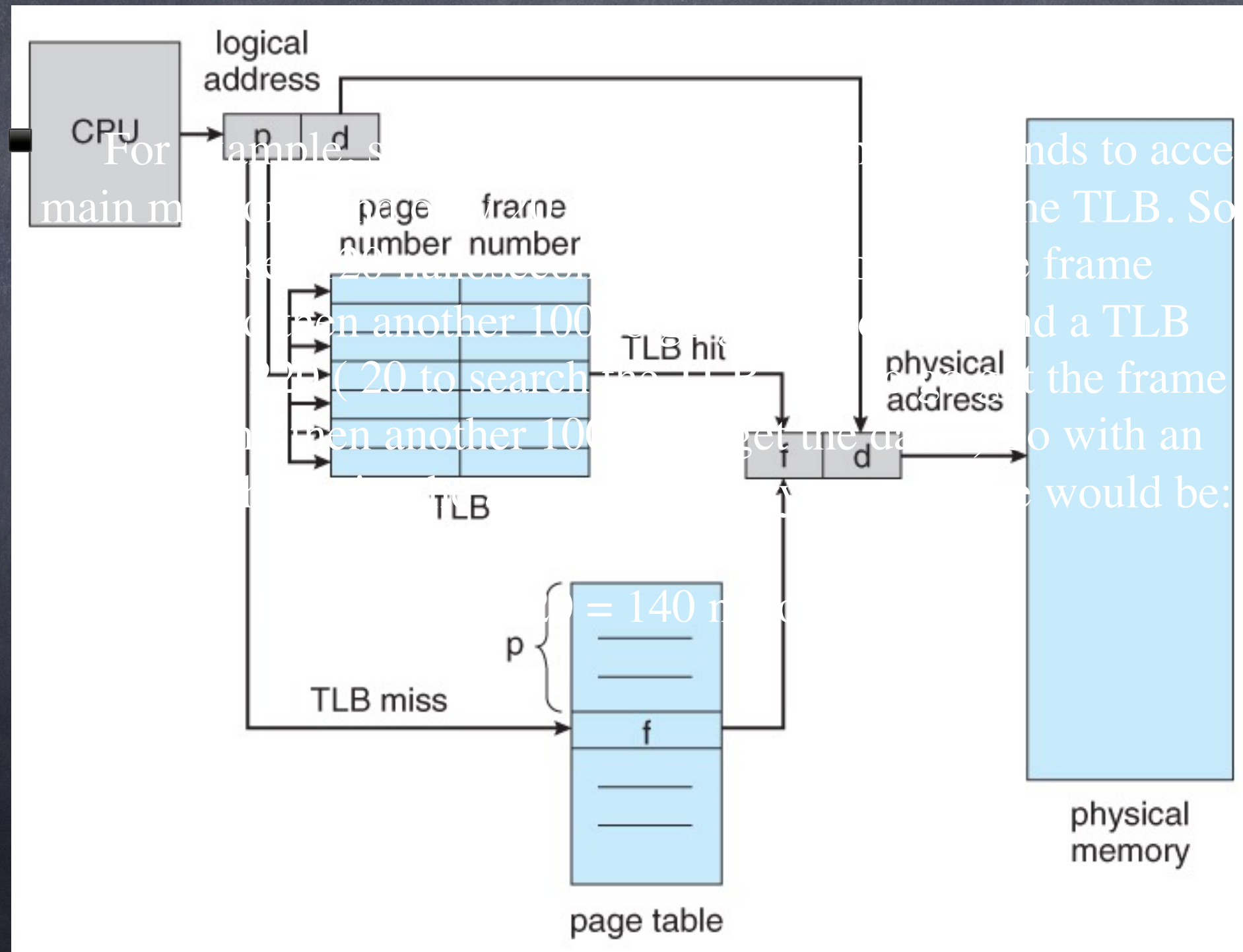
new-process page table

13
14
15
16
17
18
19
20
21



(b)

TLB = translation look-aside buffer



Shared Page

ed 1
ed 2
ed 3
data 1

process P_1

3
4
6
1

page table
for P_1

ed 1
ed 2
ed 3
data 2

process P_2

3
4
6
7

page table
for P_2

ed 1
ed 2
ed 3
data 3

process P_3

3
4
6
2

page table
for P_3

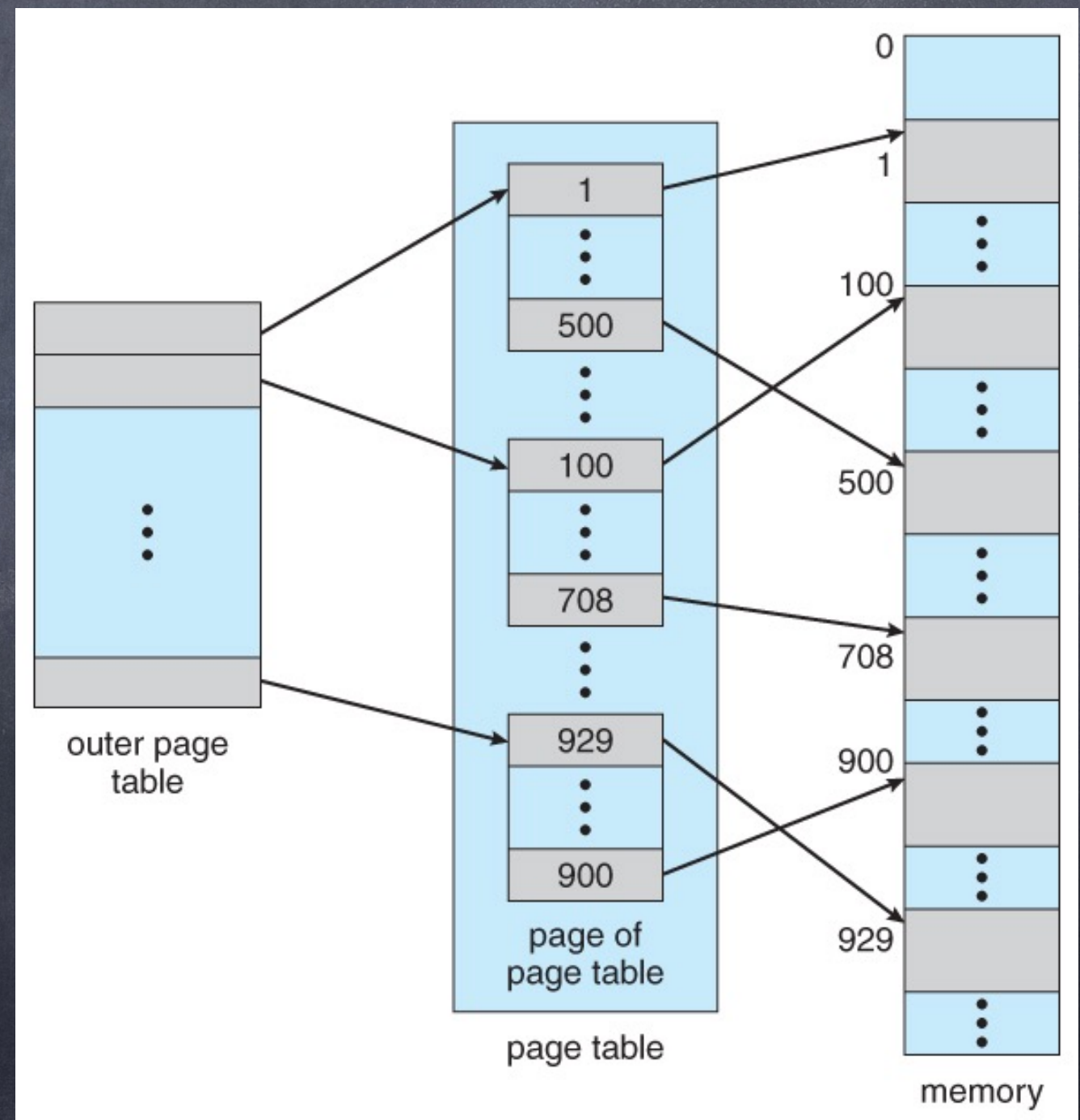
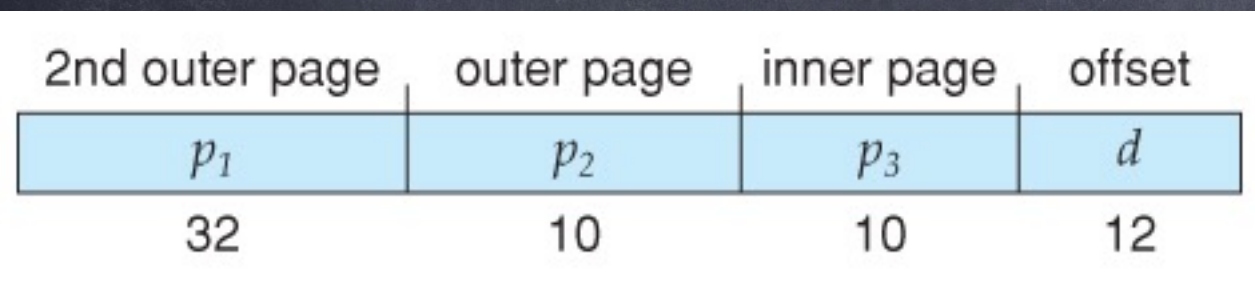
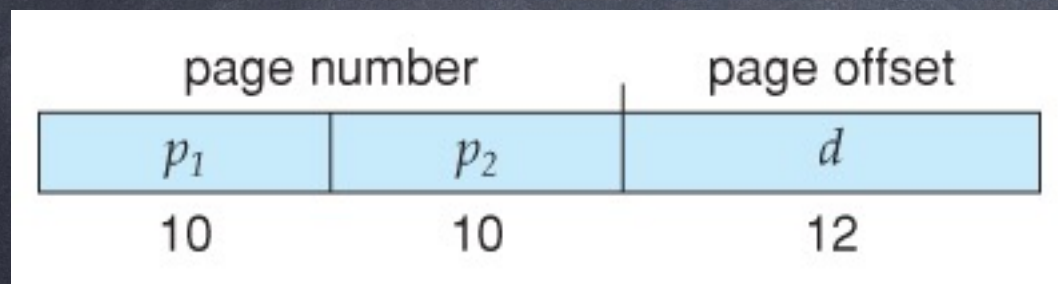
0	
1	data 1
2	data 3
3	ed 1
4	ed 2
5	
6	ed 3
7	data 2
8	
9	
10	
11	

Structure of Paging

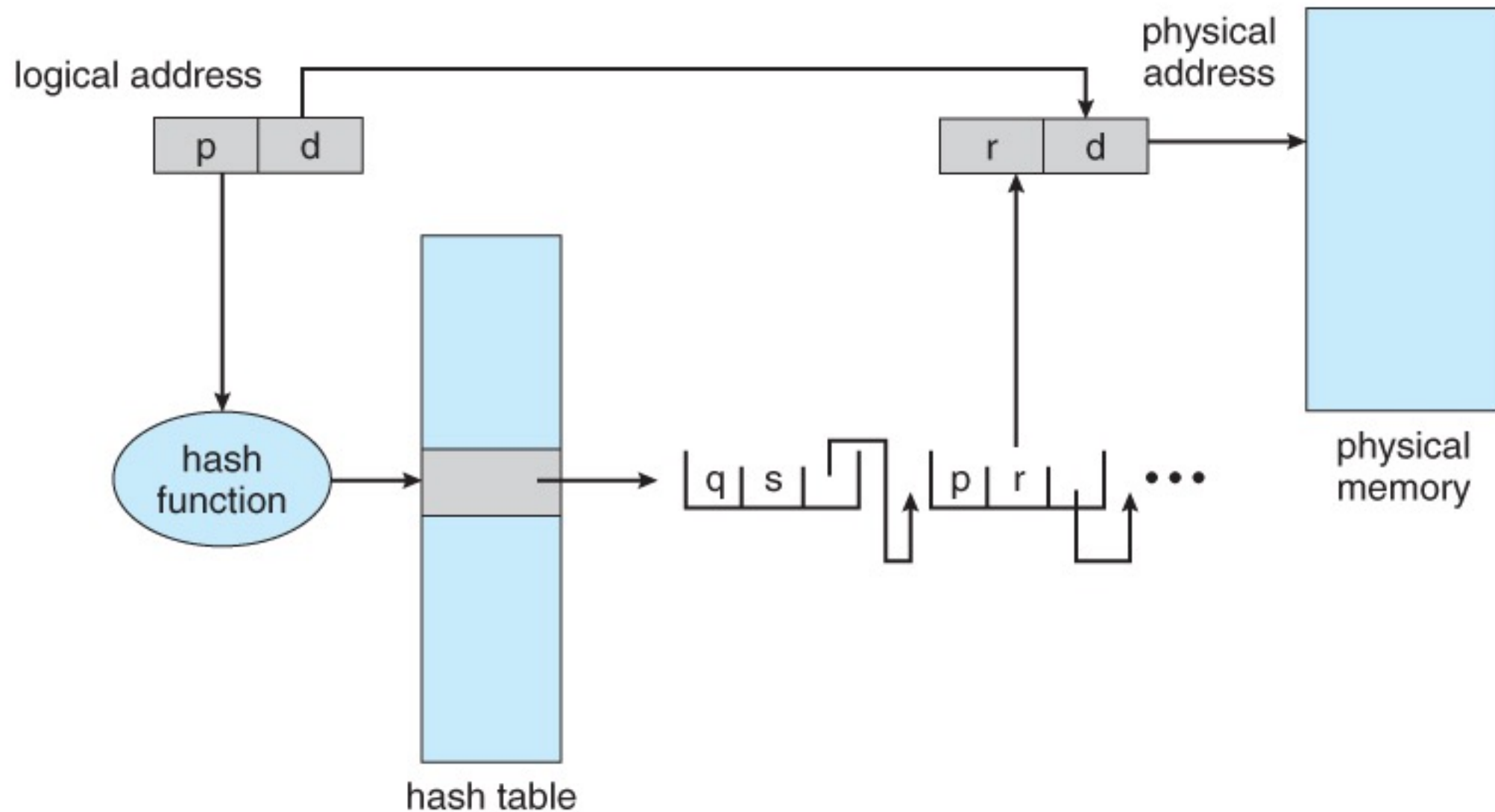
With a 2^{32} address space and 4K (2^{12}) page sizes, this leave 2^{20} entries in the page table. At 4 bytes per entry, this amounts to a 4 MB page table, which is too large to reasonably keep in contiguous memory.

- One option is to use a two-tier paging system, i.e. to page the page table.
- For example, the 20 bits described above could be broken down into two 10-bit page numbers. The first identifies an entry in the outer page table, which identifies where in memory to find one page of an inner page table. The second 10 bits finds a specific entry in that inner page table, which in turn identifies a particular frame in physical memory. (The remaining 12 bits of the 32 bit logical address are the offset within the 4K frame.)

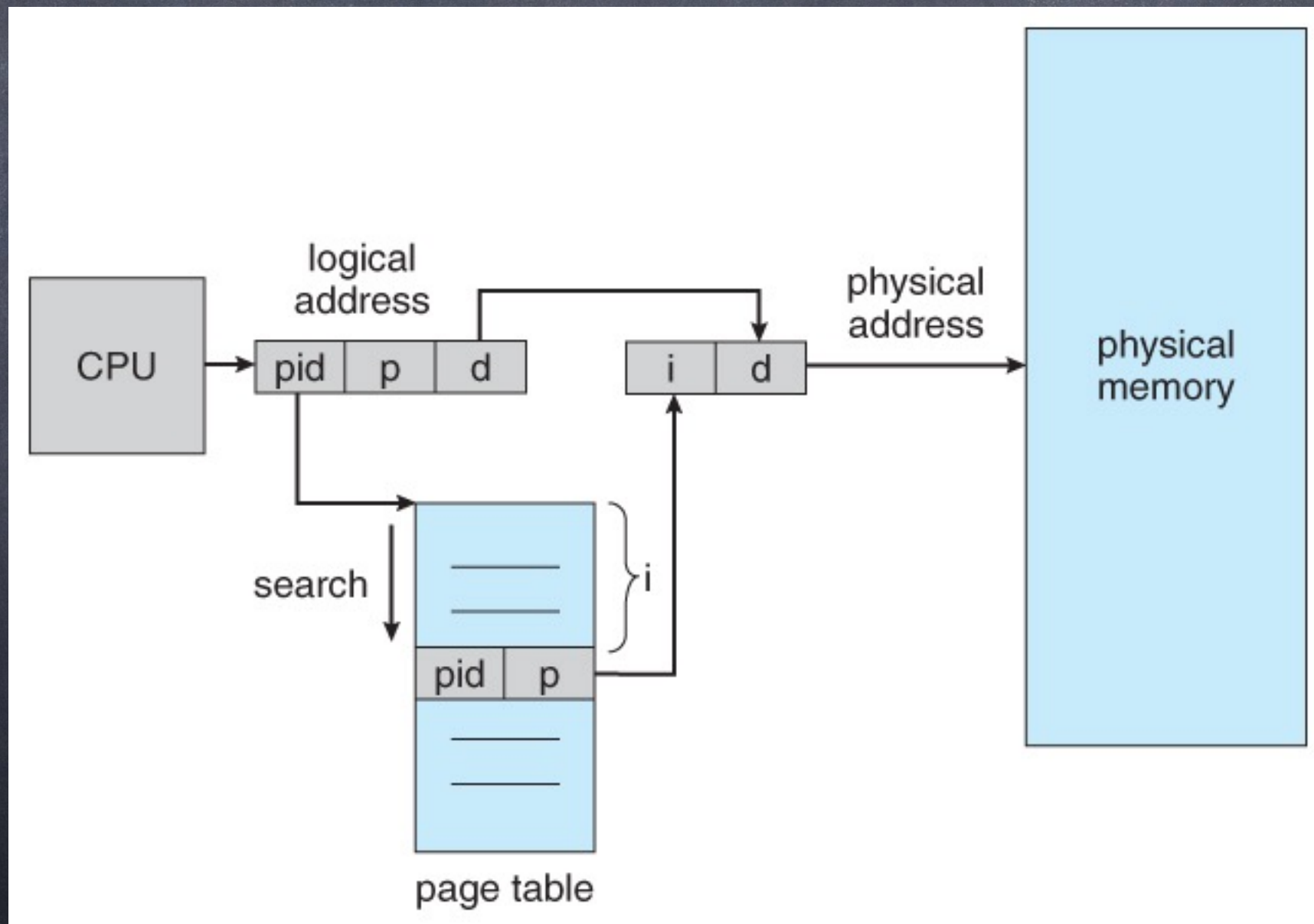
Hierarchical Paging



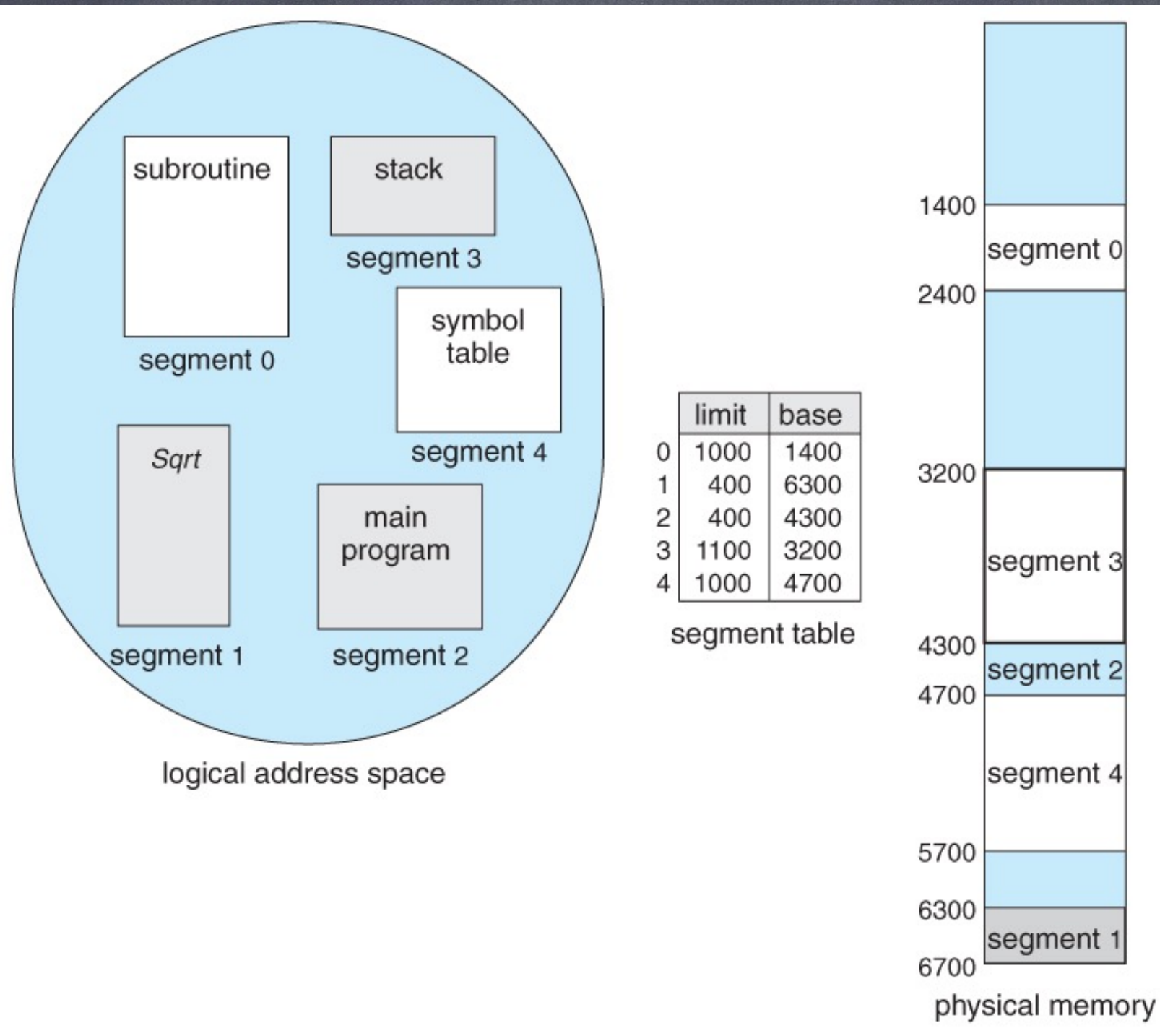
Hashed Paging



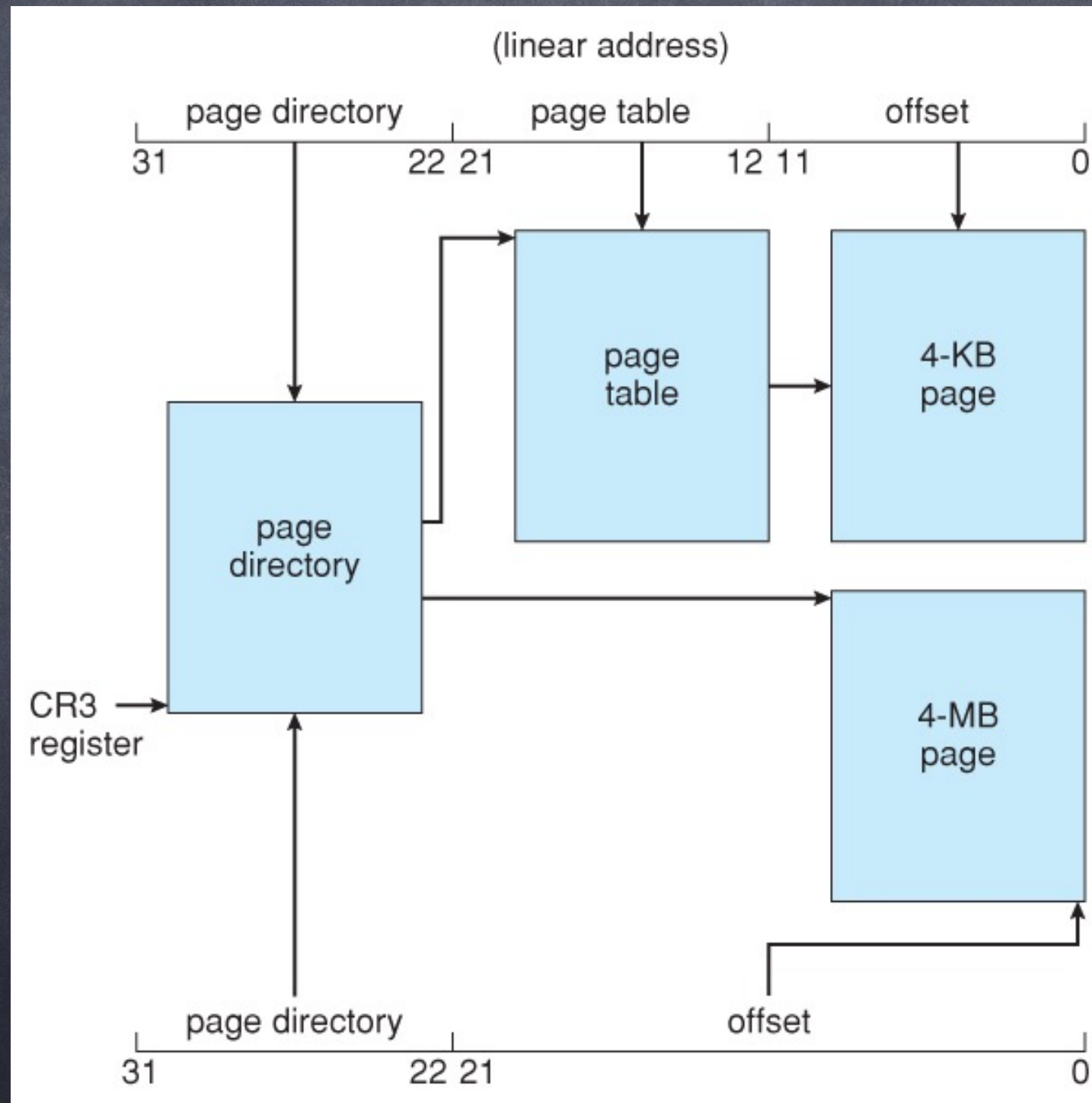
Inverted Paging



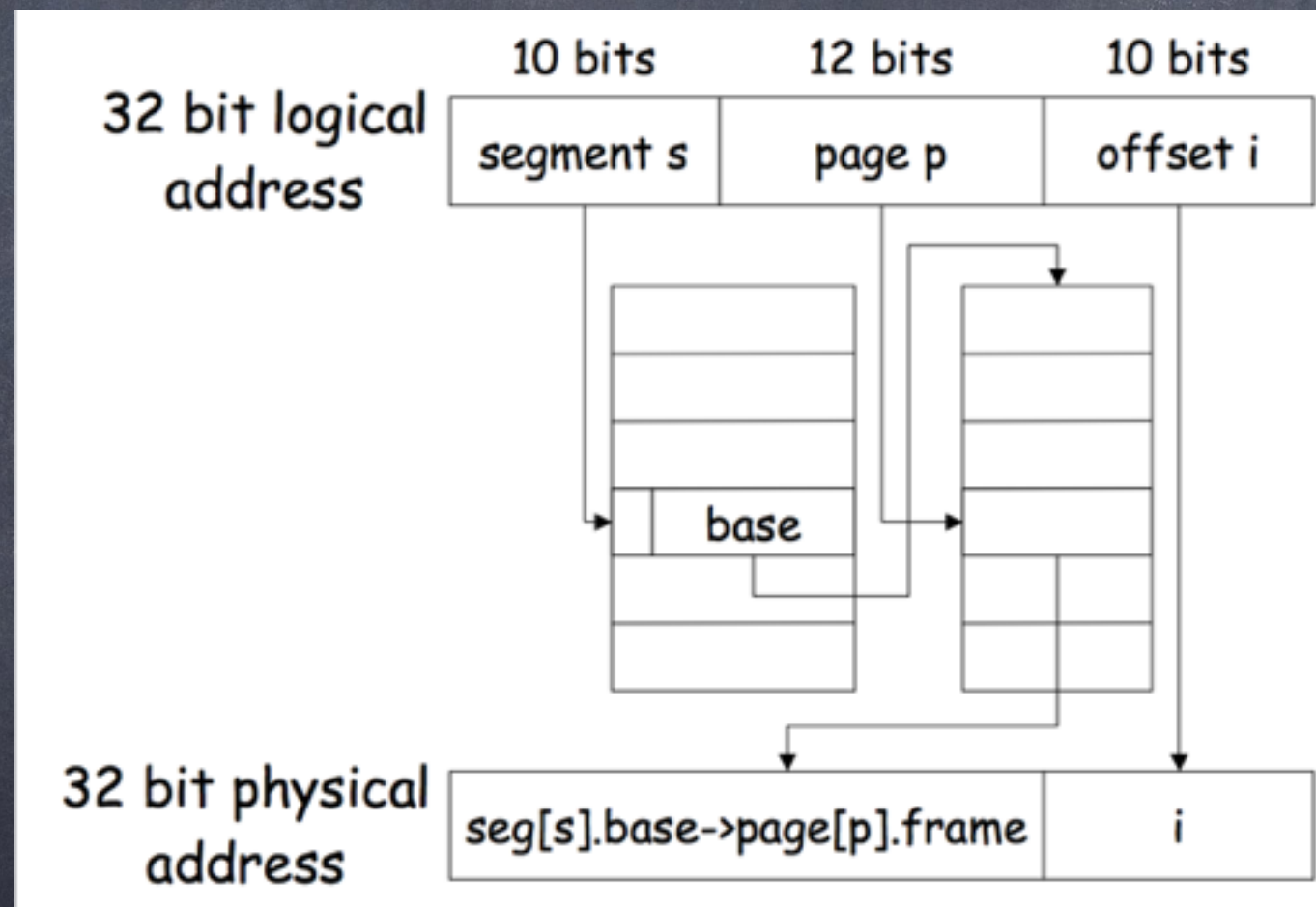
Segmentation



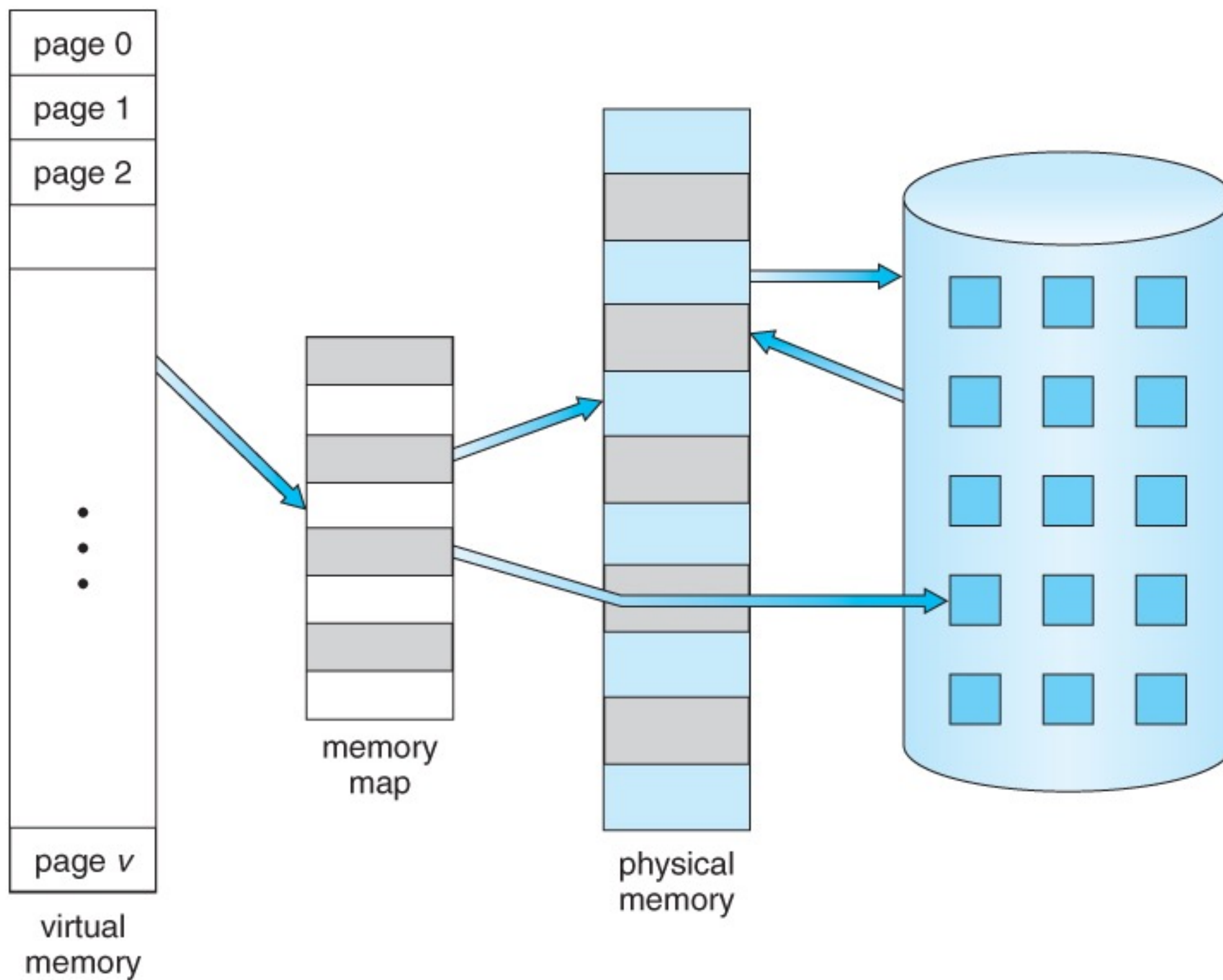
Intel CPU



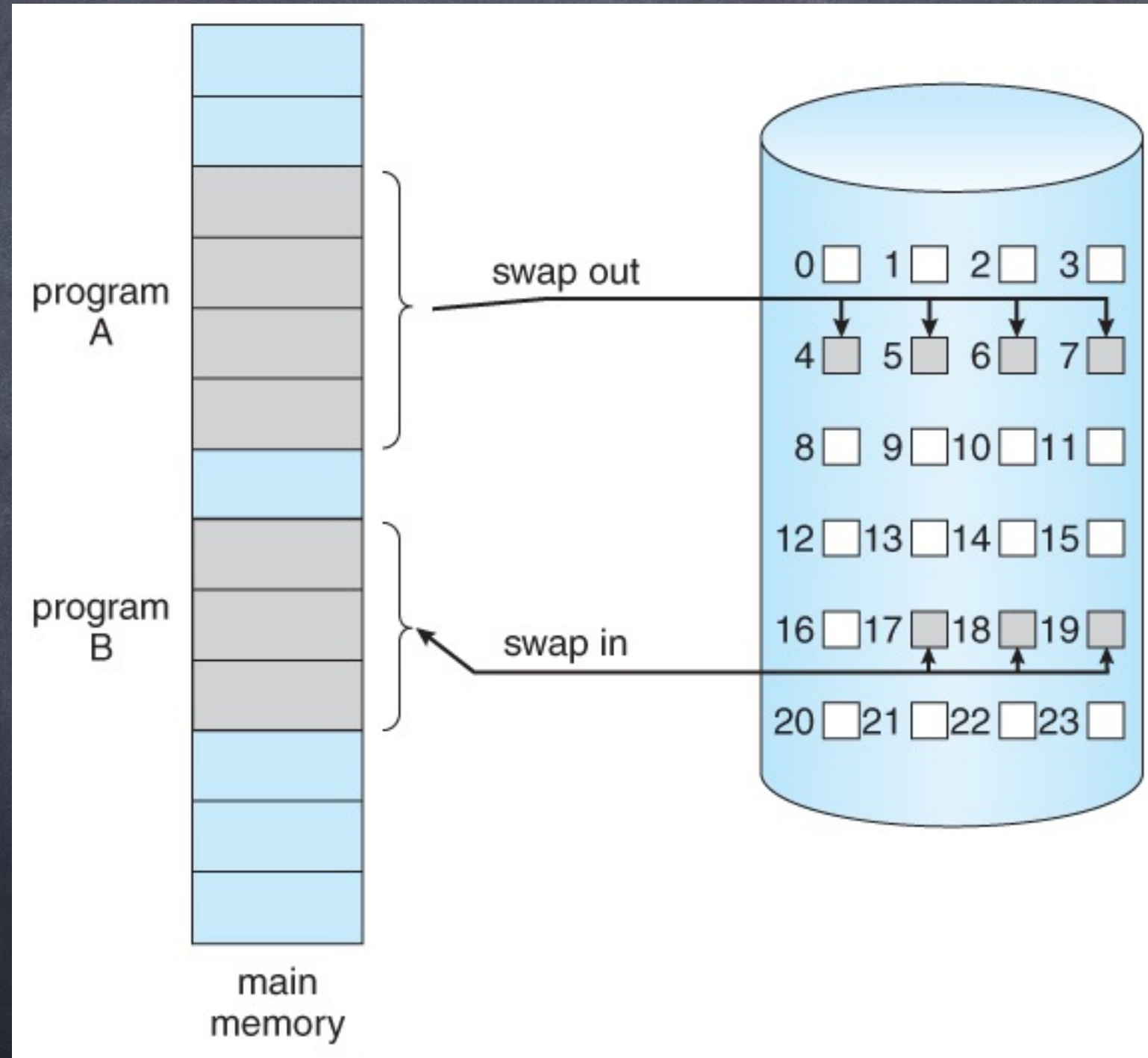
Segmentation + Paging



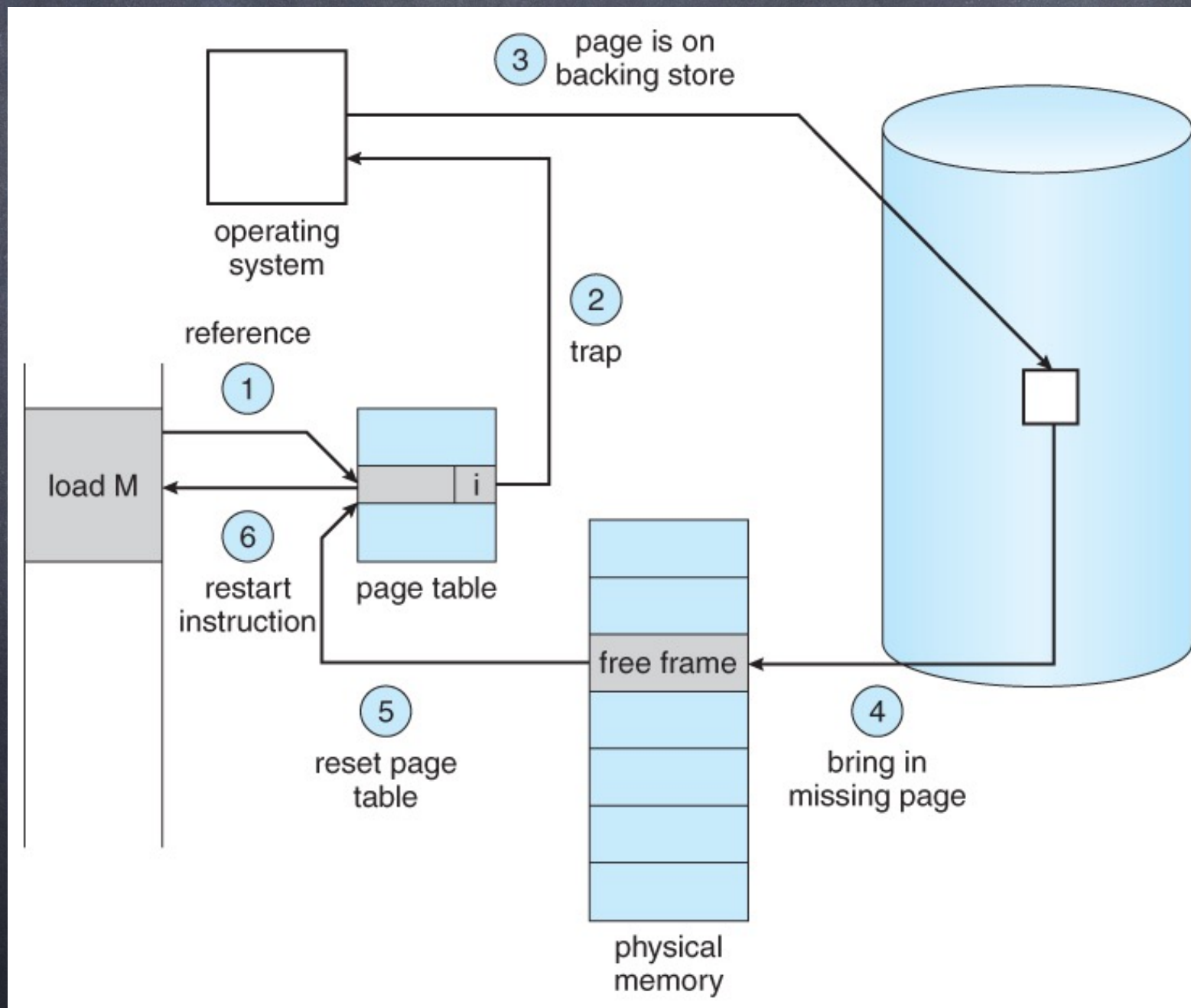
Virtual Memory



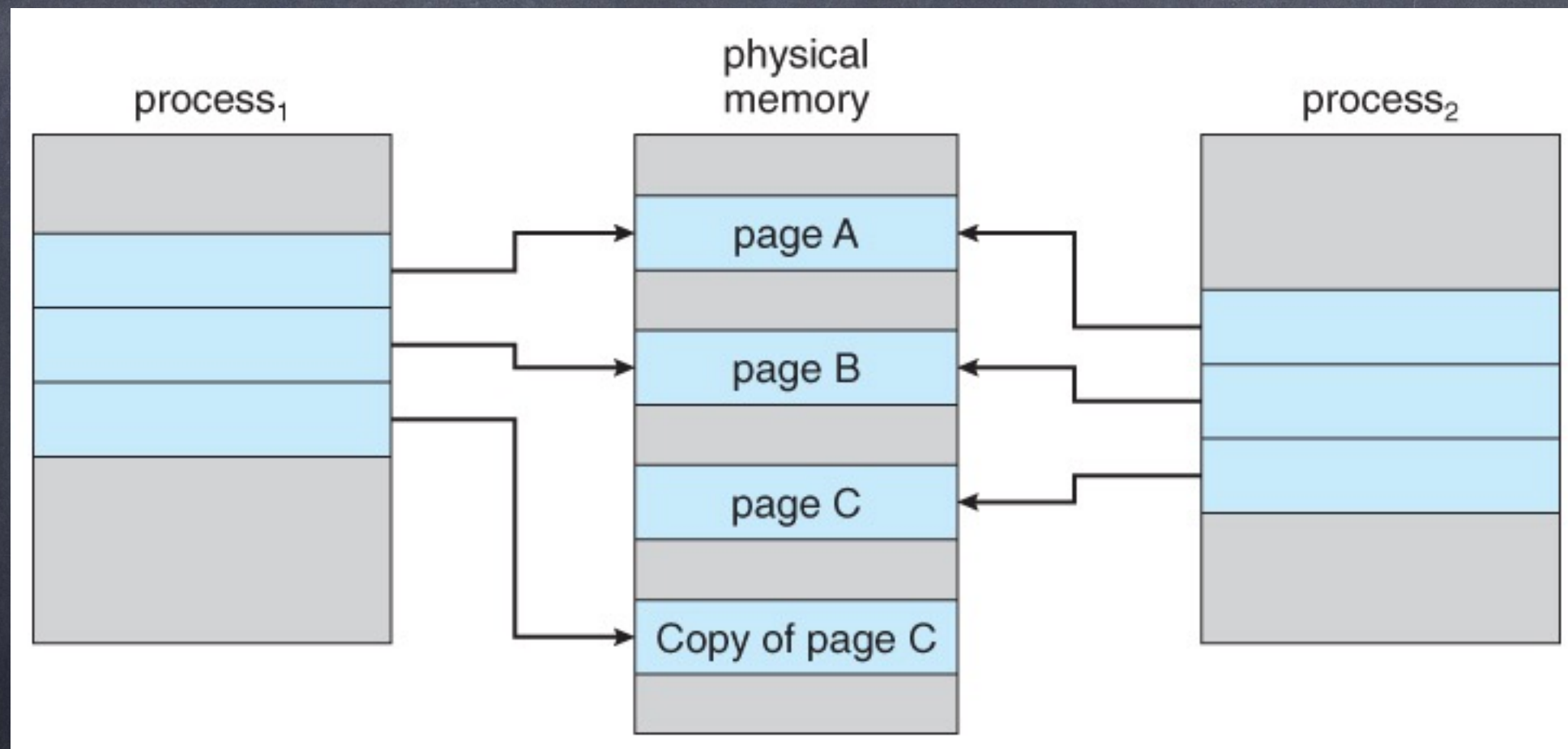
Demand Paging



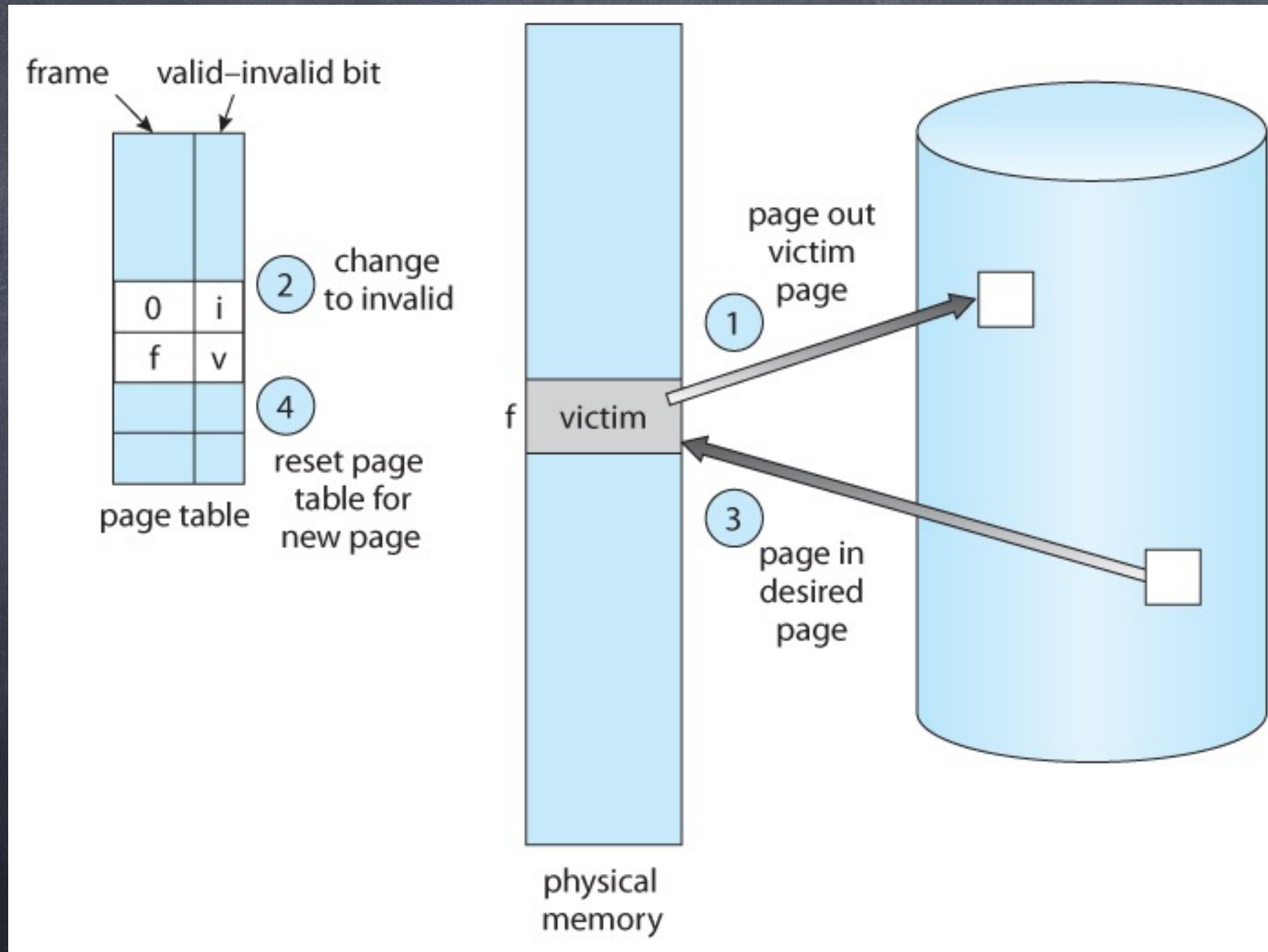
Page Fault



Copy on Write



Page Replacement



Replacement Algorithm

- FIFO
- Optimal
- LRU
- Reference Bit
- Second Chance

FIFO Page Replacement

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2																
	0	0	0																
		1	1																

page frames

Optimal Page Replacement

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2																
	0	0	0																
		1	1																

page frames

LRU Page Replacement

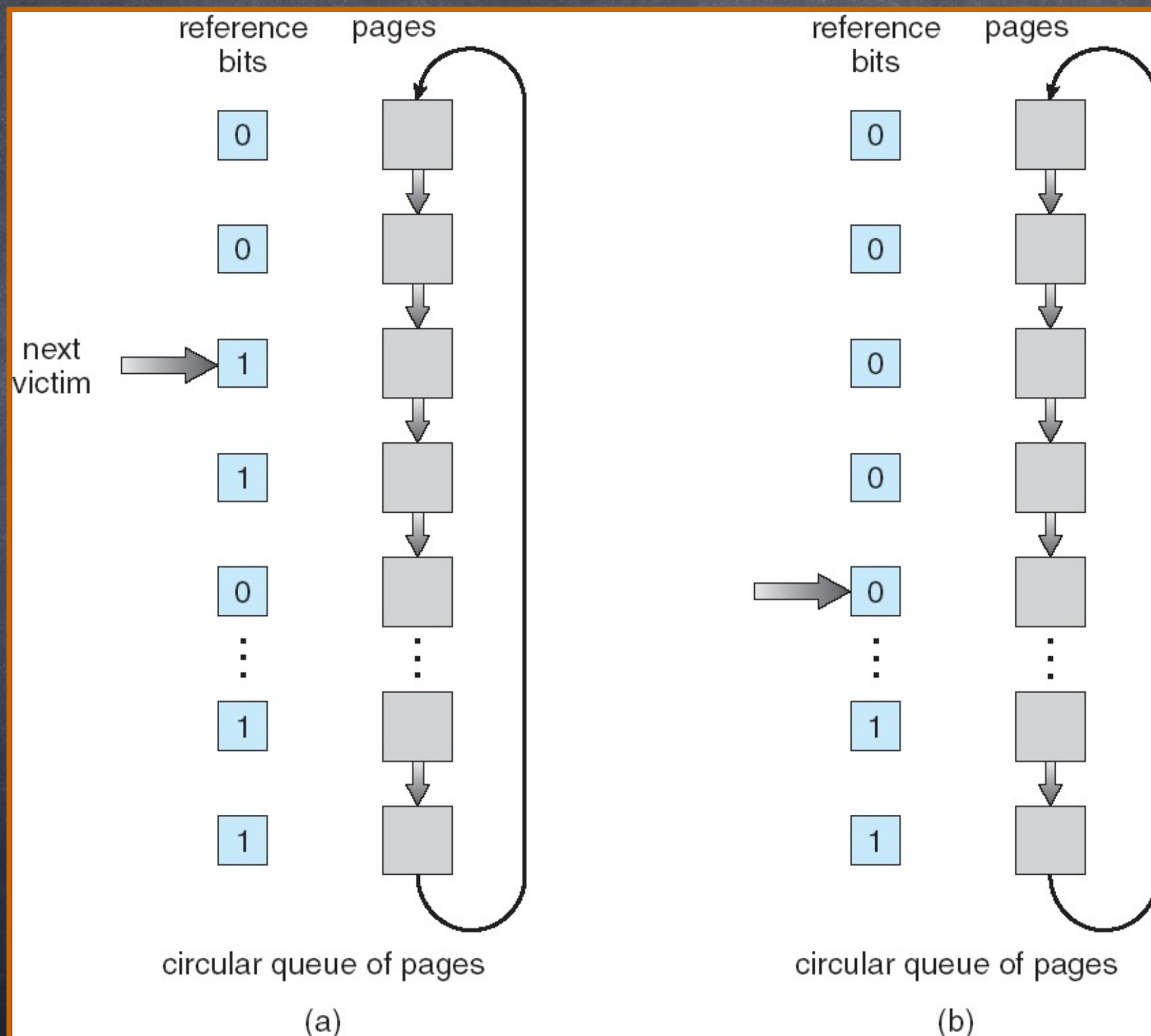
reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

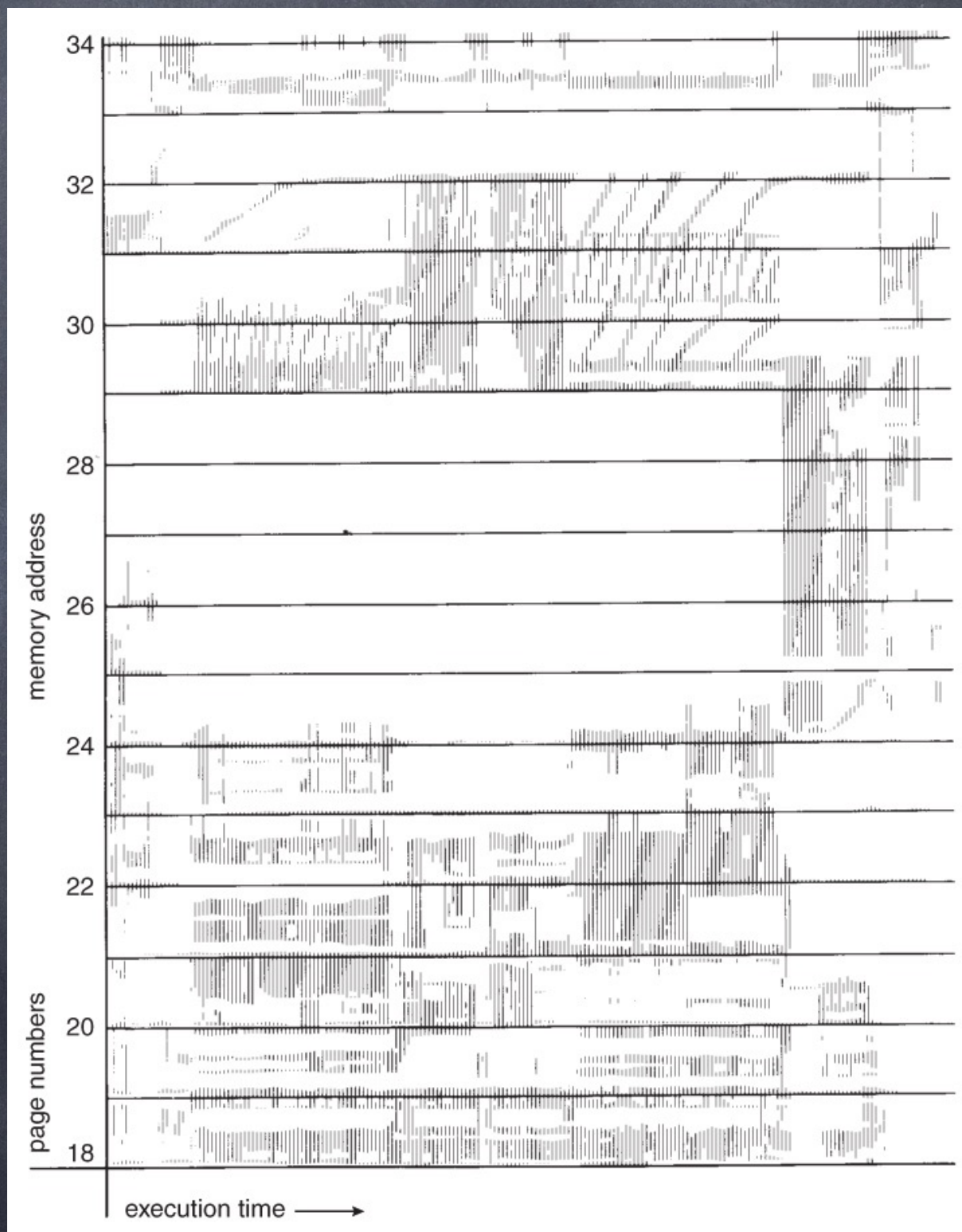
7	7	7	2		2		4	4	4	0		1		1		1
	0	0	0		0		0	0	3	3		3		0		0
		1	1		3		3	2	2	2		2		2		7

page frames

Second-Chance (clock) Page-Replacement Algorithm



Thrashing



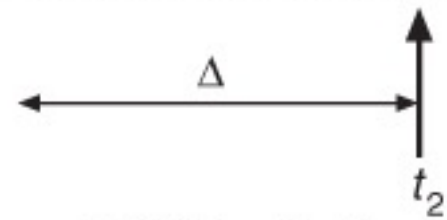
Working Set

page reference table

... 2 6 1 5 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...

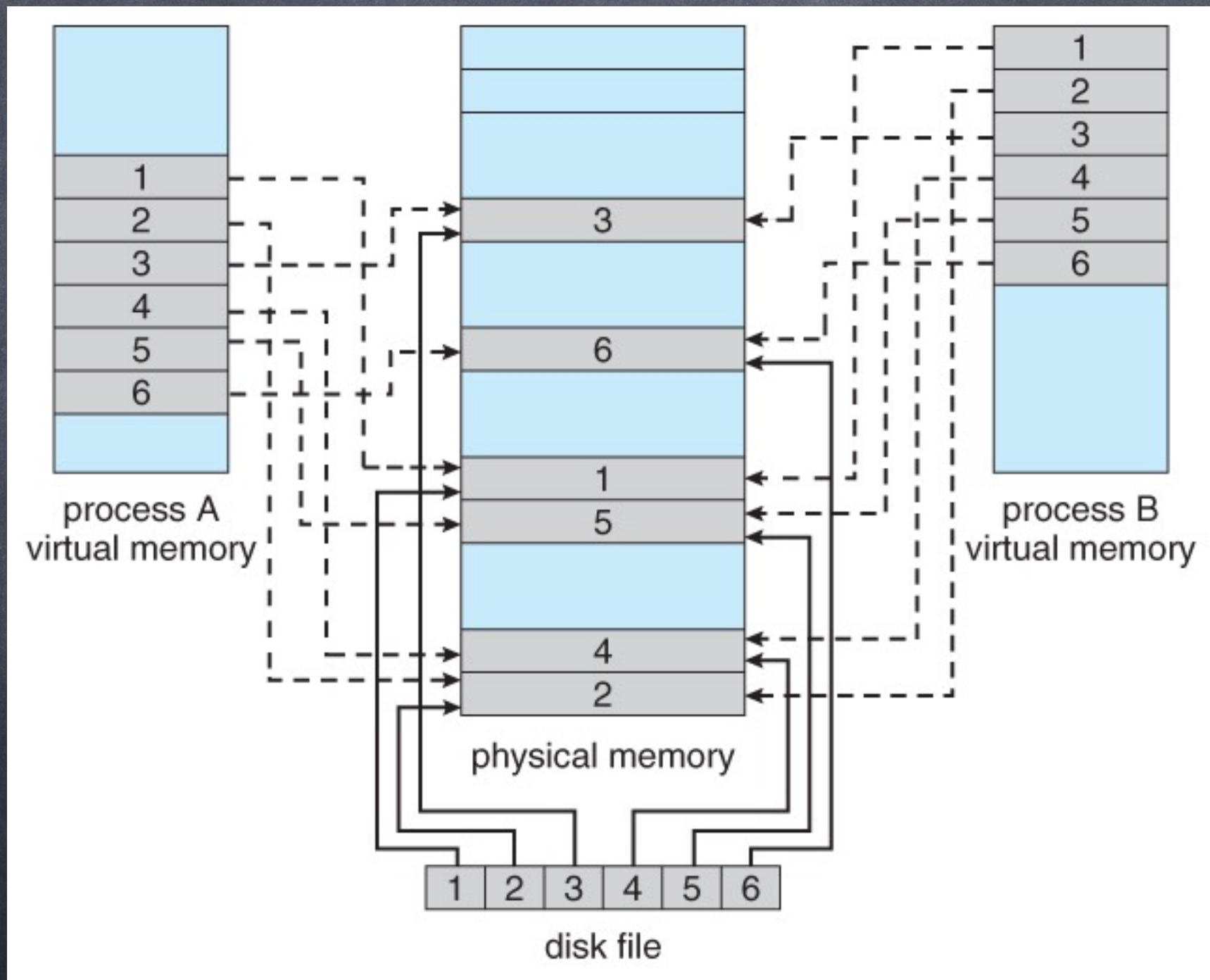


$$WS(t_1) = \{1, 2, 5, 6, 7\}$$



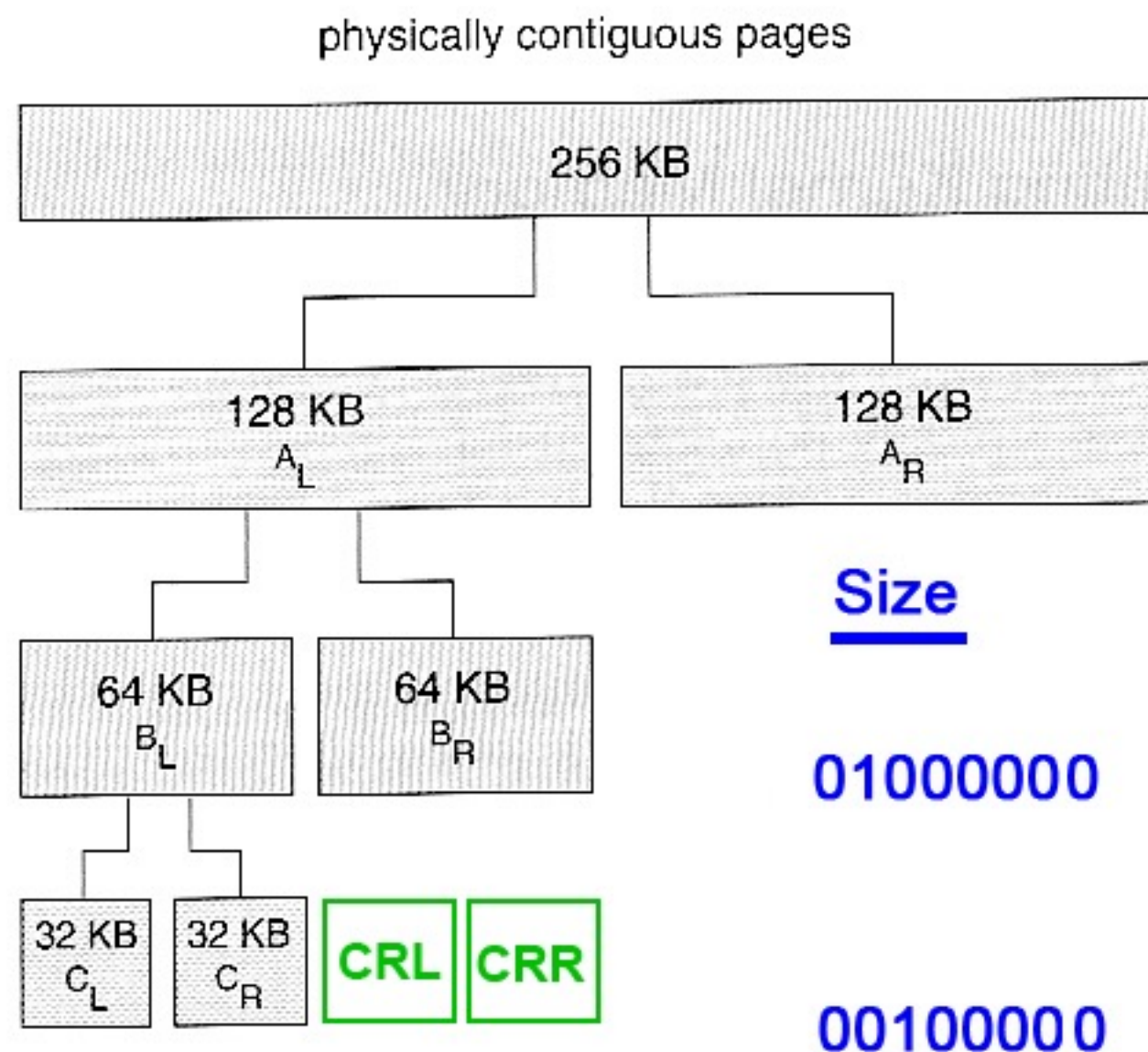
$$WS(t_2) = \{3, 4\}$$

Memory Mapped File



Kernel Memory Allocation

Buddy Allocation



Buddy Addresses

00000000

00000000

10000000

Size

01000000

00000000

01000000

00100000

00000000

00100000

01000000

01100000

Figure 9.27 Buddy system allocation.

Slab Allocation

